

P1609R0

C++ Should Support Just-in-Time Compilation

Published Proposal, 2019-03-10

This version:

<http://wg21.link/p1609r0>

Author:

[Hal Finkel](#) (Argonne National Laboratory)

Audience:

SG17

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

C++ has been specifically designed to support the creation of performance-sensitive systems. C++ templates have become a critical piece of this support, allowing the creation of algorithmic specializations to which the application can dispatch based on runtime conditions. However, many applications are pushing this capability to its limit, generating so many specializations using template instantiation that compile times are interfering with programmer productivity. In other cases, programmers simply must settle for suboptimal performance because the ahead-of-time generation of specializations with better performance is not practical given the size of the relevant parameter space. To address these challenges, this paper proposes to add to C++ the capability to support programmer-directed just-in-time (JIT) compilation.

Table of Contents

- 1 Introduction
- 2 Implementation

3 Proposal

3.1 Attribute

3.1.1 Example

3.1.2 Example

3.2 Restrictions

4 A Design Question

5 A Benchmark

5.1 Compile Time

5.2 Runtime Performance

6 Acknowledgments

§ 1. Introduction

C++ is the go-to language for programming performance-sensitive systems. In part, this is due to C++'s design philosophy of "leav[ing] no room for a lower-level language below C++ (except assembler)" (see Appendix A in p0559r0 and Bjarne's 2007 HoPL paper). As C++ programmers, however, we are faced with a fundamental dilemma:

1. A compiler can often generate more-efficient code for algorithms in which some important parameters have (compile-time) known values compared to code for the same algorithms for which those same parameters are known only during program execution. These parameters are sometimes values (e.g., the number of rows in the matrix is three) and sometimes types. Types here include both types being operated upon and types representing behavioral composition.
2. In some cases, we can template the algorithms on these relevant parameters and instantiate the templates for a large set of the parameters likely to be relevant during program execution. The program can then dispatch to a relevant instantiation during program execution, perhaps falling back to a generic implementation. However, this can have a large compile-time cost. In fact, practical limits on compile time often limit the size of the parameter space which can be covered by instantiated templates.
3. In other cases, the relevant parameter space is so large that a relevant set of instantiations cannot be feasibly selected. In such cases, we're forced to settle for a generic implementation, even if the actual sets of parameters that occur during any particular program execution is not large.

If we had the ability to instantiate templates during program execution, using parameters not known until program execution, we could provide the higher performance of fixed-parameter implementations while simultaneously providing improved compile times. This paper explores the following question: Can we naturally integrate this kind of just-in-time (JIT) compilation capability, including the ability to construct novel template instantiations during program execution, into C++?

Clearly, this kind of JIT compilation capability might not be something that all C++ implementation can provide. In some cases, this is because of technical limitation of the underlying platforms (e.g., in certain kinds of embedded systems). In other cases, this is because the additional risk factors introduced by dynamic compilation are unacceptable in certain environments. Regardless, it seems natural that capabilities in this space would fall into a conditionally-supported category.

§ 2. Implementation

The proposal below has been implemented, and that implementation is available from the repository: [github:hfinkel/llvm-project-cxxjit](https://github.com/hfinkel/llvm-project-cxxjit). There is some additional documentation on the wiki associated with the repository: [github:hfinkel/llvm-project-cxxjit/wiki](https://github.com/hfinkel/llvm-project-cxxjit/wiki). The data presented below was generated using this implementation.

§ 3. Proposal

This proposal is designed to be minimal, and proposes a JIT capability with an interface limited to function templates. JIT compilation engines, as a practical matter, are generally designed to provide new functions (e.g., to provide a function pointer to some newly-compiled function), making this a natural fit to the underlying technology. Moreover, this allows us to elide questions regarding partial specialization.

§ 3.1. Attribute

Function templates can be tagged for just-in-time compilation by using the attribute:

```
[[jit]]
```

[*Note:*

In the aforementioned implementation of this proposal, the attribute is named `[[clang::jit]]`.

-- end note]

The attributed function template provides for additional features and restrictions. Features:

1. Instantiations of this function template will not be constructed at compile time, but rather calling a specialization of the template, or taking the address of a specialization of the template, will trigger the instantiation and compilation of the template during program execution. Note that this property is non-normative (i.e., not observable within the abstract machine).
2. Non-constant expressions may be provided for the non-type template parameters, and these values will be used during program execution to construct the type of the requested instantiation.

§ 3.1.1. Example

```
#include <iostream>
#include <cstdlib>

template <int x>
[[jit]] void run() {
    std::cout << "I was compiled at runtime, x = " << x << "\n";
}

int main(int argc, char *argv[]) {
    int a = std::atoi(argv[1]);
    run<a>();
}
```

3. Type arguments to the template can be provided as strings. If the argument is implicitly convertible to a `const char *`, then that conversion is performed, and the result is used to identify the requested type. Otherwise, if an object is provided, and that object has a member function named `c_str()`, and the result of that function can be converted to a `const char *`, then the call and conversion (if necessary) are performed in order to get a string used to identify the type. The string is parsed and analyzed to identify the type in the declaration context of the parent to the function triggering the instantiation. Whether types defined after the point in the source code that triggers the instantiation are available is not specified.

§ 3.1.2. Example

```
#include <iostream>

struct F {
    int i;
    double d;
};

template <typename T, int S>
struct G {
    T arr[S];
};

template <typename T>
[[jit]] void run() {
    std::cout << "I was compiled at runtime, sizeof(T) = " << sizeof(T) << "\n";
}

int main(int argc, char *argv[]) {
    std::string t(argv[1]);
    run<t>();
}
```

§ 3.2. Restrictions

1. Because the body of the template is not instantiated at compile time, `decltype(auto)` and any other type-deduction mechanisms depending on the body of the function are not available.
2. Because the template specializations are not compiled until during program execution, they're not available at compile time for use as non-type template arguments, etc.

Explicit specializations of a JIT function template are not just-in-time compiled.

Note: A JIT template with a pointer/reference non-type template parameter which is provided with a runtime pointer value will generate a different instantiation for each pointer value. If the pointer provided points to a global object, no attempt is made to map that pointer value back to the name of the global object when constructing the new type.

Note: In general, pointer/reference-type non-type template arguments are not permitted to point to subobjects. This restriction still applies formally to the templates instantiated at runtime using runtime-provided pointer values. This has important optimization benefits: pointers that can be traced back to distinct underlying objects are known not to alias, and these template parameters appear to the optimizer to have this unique-object property.

§ 4. A Design Question

The proposal above reflects what has been implemented, but the use of a attribute might be suboptimal from a language-design perspective. Some points against this attribute approach:

1. The template is not otherwise special, it's the point of instantiation that's special (and that might not compile at all without the JIT support).
2. The uses of the template look vaguely normal, and so places where you might invoke the JIT compiler will be difficult to spot during code review.
3. The current mechanism provides no place to get out an error or provide a fall-back execution path - except that having the runtime throw an exception might work.

Thus, it might be better to use, e.g., a keyword near the point of instantiation. Perhaps something like this:

```
jit_this_template foo<argc>();
```

or maybe:

```
foo jit_this_template <argc>();
```

where `jit_this_template` is a new keyword.

The disadvantage of tying the JIT use to the point of instantiation instead of to the template itself, is that we need to decide that happens if the same instantiation is created both at compile time in the usual manner and also requested to be delayed until program execution. This might be okay or it might be some kind of ODR violation.

§ 5. A Benchmark

As a benchmark to illustrate the feature, we'll adapt a benchmark from the [Eigen](https://github.com/eigenteam/eigen-git-mirror/blob/master/bench/benchmark.cpp) library. Specifically, this one: <https://github.com/eigenteam/eigen-git-mirror/blob/master/bench/benchmark.cpp>

We want to look at two aspects: Compile time and runtime performance. Eigen provides a matrix type which can either have compile-time-specific or runtime-specified sizes (i.e., the number of rows and columns).

```
#include <iostream>
#include <string>
#include <chrono>
#include <cstdlib>

#include <Eigen/Core>

using namespace std;
using namespace Eigen;
```

If we wish to support a variant of this benchmark supporting float, double, and long double, and supporting any size at runtime, we can adapt the code as:

```

template <typename T>
void test_aot(int size, int repeat) {
    Matrix<T,Dynamic,Dynamic> I = Matrix<T,Dynamic,Dynamic>::Ones(size, size);
    Matrix<T,Dynamic,Dynamic> m(size, size);
    for(int i = 0; i < size; i++)
        for(int j = 0; j < size; j++) {
            m(i,j) = (i+size*j);
        }

    auto start = chrono::system_clock::now();

    for (int r = 0; r < repeat; ++r) {
        m = Matrix<T,Dynamic,Dynamic>::Ones(size, size) + T(0.00005) * (m + (m^3));
    }

    auto end = chrono::system_clock::now();
    cout << "AoT: " << chrono::duration<double>(end - start).count() << " s\r\n";
}

void test_aot(std::string &type, int size, int repeat) {
    if (type == "float")
        test_aot<float>(size, repeat);
    else if (type == "double")
        test_aot<double>(size, repeat);
    else if (type == "long double")
        test_aot<long double>(size, repeat);
    else
        cout << type << "not supported for AoT\n";
}

```

To do the same thing with the JIT feature, we can write:


```

template <typename T, int size>
[[jit]] void test_jit_sz(int repeat) {
    Matrix<T,size,size> I = Matrix<T,size,size>::Ones();
    Matrix<T,size,size> m;
    for(int i = 0; i < size; i++)
        for(int j = 0; j < size; j++) {
            m(i,j) = (i+size*j);
        }

    auto start = chrono::system_clock::now();

    for (int r = 0; r < repeat; ++r) {
        m = Matrix<T,size,size>::Ones() + T(0.00005) * (m + (m*m));
    }

    auto end = chrono::system_clock::now();
    cout << "JIT: " << chrono::duration<double>(end - start).count() << " s\r\n";
}

void test_jit(std::string &type, int size, int repeat) {
    return test_jit_sz<type, size>(repeat);
}

```

And we can use very-similar code to construct explicit instantiations at compile time, but of course, then we're limited to support for the explicit sizes we have selected.

§ 5.1. Compile Time

Compiling using the implementation linked above on an Intel Xeon E5-2699 using the flags `-march=native -ffast-math -O3`, and measuring compile time using "user" time from the Linux `time` command.

	Time	Time over Base
JIT Only	3.5s	0.92s
(AoT) Single Specialization (double, size = 16)	4.95s	2.37s
(AoT) Single Specialization (double, size = 7)	3.3s	0.72s
(AoT) Single Specialization (double, size = 3)	3.2s	0.62s
(AoT) Single Specialization (double, size = 1)	2.95s	0.37s
(AoT) Two Specializations (double, size = 16) and (double, 7)	5.7s	3.12s

	Time	Time over Base
Generic AoT Only (three floating-point types with dispatch)	9.7s	7.12s
Generic AoT Only (double only)	5.3s	2.72s
Nothing (just the includes and a main function)	2.58s	-

As you can see, the time for generating each specific specialization is essentially additive, and they get more expensive as the fixed matrix sizes get longer. Generating the code for the JIT has a compile-time cost, but it's not even as expensive as a single non-fixed-size implementation.

§ 5.2. Runtime Performance

For (double, size = 3); a repeat count of 40000000. Times as reported by the code (excludes JIT compilation time).

	Time
JIT	1.0s
Single Specialization	1.01s
AoT	8.05s

For (double, size = 7)

	Time
JIT	8.34s
Single Specialization	8.45s
AoT	20s

For (double, size = 16)

	Time
JIT	35.3s
Single Specialization	35.1s
AoT	36.2s

A few trends to notice:

The JIT-generated code is **significantly** faster than the ahead-of-time-generated code for small matrix sizes. The advantage becomes less significant as the matrix sizes become larger.

Thus, using the JIT gives the performance advantages of using many ahead-of-time specializations, and is sometimes even better, with very low compile-time cost.

§ 6. Acknowledgments

I'd like to thank David Poliakoff for a lot of testing and feedback on the implementation, and I'd like to think the many committee members who provided me with feedback on this idea during the Kona meeting.

This research was supported by the Exascale Computing Project (17-SC-20-SC), a collaborative effort of two U.S. Department of Energy organizations (Office of Science and the National Nuclear Security Administration) responsible for the planning and preparation of a capable exascale ecosystem, including software, applications, hardware, advanced system engineering, and early testbed platforms, in support of the nation's exascale computing imperative. Additionally, this research used resources of the Argonne Leadership Computing Facility, which is a DOE Office of Science User Facility supported under Contract DE-AC02-06CH11357.